🎵 K-pop English

BTS 가사로 배우는 영어 학습 앱

프로젝트 정의서 (Project Specification Document)

프로젝트명: K-pop English

버전: 1.0.0

작성일: 2026년 3월 25일

기술 스택: React Native, Expo, TypeScript, Spotify API

플랫폼: iOS, Android, Web

GitHub: https://github.com/ymi-hub/K-pop

목차

1. 프로젝트 개요
2. 시스템 아키텍처
3. 기술 스택 및 의존성
4. API 연동 사양
5. 인증 및 보안
6. 데이터 흐름도
7. 화면 구조 및 메뉴
8. 컴포넌트 명세
9. 환경 설정
10. 설치 및 실행 가이드
11. 에러 처리 및 제한사항
12. 향후 개선 계획

1. 프로젝트 개요

1.1 목적

BTS의 영문 가사를 활용하여 영어를 학습할 수 있는 모바일 애플리케이션 개발

1.2 주요 기능

- **Spotify 연동:** BTS 전체 곡 목록 조회 및 30초 미리듣기
- **실시간 가사 싱크:** 재생 시간에 맞춰 가사 표시
- **영어 단어 학습:** 가사 속 단어 클릭 시 사전 검색
- **크로스 플랫폼 지원:** iOS, Android, Web 동시 지원
- **OAuth 인증:** Spotify 계정 연동

1.3 대상 사용자

- BTS 팬
- 영어 학습자
- 음악으로 언어를 배우고 싶은 사용자

1.4 프로젝트 범위

항목	범위
음악 소스	Spotify (30초 미리듣기)
가사 소스	lrclib.net API
사전 API	Free Dictionary API
인증	Spotify OAuth 2.0 PKCE

배포 플랫폼	iOS, Android (Expo), Web

2. 시스템 아키텍처

2.1 전체 아키텍처

```
┌─────────────────────────────────────────────────────────────┐ │ 클라이언트 (React
Native) │ │ ┌──────────────────────────────────────────────────────────┐ │ │ │ UI Layer
(화면 & 컴포넌트) │ │ │ ├──────────────────────────────────────────────────────────┤ │ │ │
HomeScreen │ PlayerScreen │ LoginScreen │ │ │
└──────────────────────────────────────────────────────────┘ │ │ ↓ │ │
┌──────────────────────────────────────────────────────────┐ │ │ │ Service Layer (비즈니스
로직) │ │ │ ├──────────────────────────────────────────────────────────┤ │ │ │ spotify.ts
│ lyrics.ts │ spotifyAuth.ts │ │ │
└──────────────────────────────────────────────────────────┘ │ │ ↓ │ │
┌──────────────────────────────────────────────────────────┐ │ │ │ Storage Layer (상태 관
리) │ │ │ ├──────────────────────────────────────────────────────────┤ │ │ │
AsyncStorage (토큰) │ React State (UI 상태) │ │ │
└──────────────────────────────────────────────────────────┘ │
└─────────────────────────────────────────────────────────────┘ ↓
┌─────────────────────────────────────────────────────────────┐ │ 외부 API (서버) │
├─────────────────────────────────────────────────────────────┤ │ Spotify API │
lrclib.net │ Free Dictionary API │
└─────────────────────────────────────────────────────────────┘
```

2.2 계층 설명

UI Layer

- 사용자가 상호작용하는 화면
- React Native 컴포넌트로 구성
- iOS/Android/Web 동시 지원

Service Layer

- API 호출 및 비즈니스 로직 처리
- 데이터 변환 및 가공
- 인증 및 토큰 관리

Storage Layer

- AsyncStorage: Spotify 토큰 저장
- React State: UI 상태 관리

3. 기술 스택 및 의존성

3.1 핵심 의존성

패키지명	버전	용도
react-native	0.83.2	모바일 UI 프레임워크
expo	~55.0.8	개발 플랫폼
expo-av	16.0.8	오디오/비디오 재생
@react-navigation/native	7.1.34	화면 네비게이션
@react-native-async-storage/async-storage	최신	로컬 저장소
firebase	12.11.0	백엔드 서비스 (선택)
TypeScript	~5.9.2	정적 타입 언어
expo-linear-gradient	55.0.9	UI 그래디언트
expo-blur	55.0.10	UI 블러 효과
@expo/vector-icons	15.1.1	아이콘 라이브러리

3.2 개발 의존성

패키지명	버전
@types/react	~19.2.2
typescript	~5.9.2

4. API 연동 사양

4.1 Spotify Web API

> 📌 **목적**
>
> BTS 곡 목록 조회 및 메타데이터 획득

주요 엔드포인트

메서드	엔드포인트	설명
GET	/artists/{id}/albums	아티스트 앨범 조회
GET	/albums/{id}/tracks	앨범 트랙 조회
GET	/search	곡 검색

인증

```
Authorization: Bearer {access_token}
```

응답 데이터 예시

```
{ "id": "3mAF4MXWFCxCDBRZcEPr98", "name": "Spring Day", "artists": [{"name":
"BTS"}], "preview_url": "https://...", "duration_ms": 245573, "album":
{"images": [{"url": "..."}]} }
```

4.2 lrclib.net API (가사)

> 📌 **목적**
>
> 타임스탬프 포함 동기화된 가사 조회

엔드포인트

```
GET https://lrclib.net/api/search?track_name=Spring%20Day&artist_name=BTS
```

응답 필드

필드	타입	설명
syncedLyrics	string	[mm:ss.xx]text 형식의 동기화 가사
plainLyrics	string	타임스탬프 없는 일반 가사

4.3 Free Dictionary API (단어 사전)

> 📌 **목적**
>
> 영어 단어 정의 및 발음 조회

엔드포인트

```
GET https://api.dictionaryapi.dev/api/v2/entries/en/{word}
```

응답 샘플

```
{ "word": "spring", "phonetic": "/sprɪŋ/", "meanings": [{ "partOfSpeech":
"verb", "definitions": [{ "definition": "move rapidly upward...", "example":
"he sprang up..." }] }] }
```

5. 인증 및 보안

5.1 Spotify OAuth 2.0 PKCE 플로우

```
1. 사용자 로그인 요청 ↓ 2. Code Verifier 생성 ↓ 3. Code Challenge 생성 (SHA-256) ↓ 4. Authorization URL로 리다이렉트 ↓ 5. 사용자 승인 ↓ 6. Authorization Code 수신 ↓ 7. Code Verifier와 함께 Token 요청 ↓ 8. Access Token 획득 ↓ 9. AsyncStorage에 토큰 저장
```

5.2 토큰 관리

저장 위치

- **AsyncStorage:** spotify_token (크로스 플랫폼)
- **저장 정보:** access_token + expiry_time

토큰 갱신 정책

- 만료 시간 체크: `Date.now() > expiry_time`
- 만료된 토큰: 자동 삭제 및 재인증 필요

5.3 보안 고려사항

항목	적용 현황
PKCE 사용	✅ 적용됨
HTTPS	✅ Spotify API 강제
Client Secret 없음	✅ PKCE로 대체
환경 변수 분리	✅ .env 파일 사용

⚠️ **주의:** .env 파일이 .gitignore에 포함되어 있으므로 민감한 정보가 저장소에 노출되지 않습니다.

6. 데이터 흐름도

6.1 앱 초기 시작 흐름

```
App.tsx (마운트) ↓ initializeApp() 호출 ↓ getStoredToken() - AsyncStorage에서 토큰 확인 ↓ 토큰 있음? ├─ YES → setToken() → HomeScreen 이동 └─ NO → LoginScreen 표시
```

6.2 곡 선택 후 재생 흐름

```
HomeScreen: 곡 선택 ↓ handleSelectTrack(track) ↓ 1. 현재 트랙 설정 2. PlayerScreen으로 이동 3. getLyrics(trackName, artistName) 호출 ├─ lrclib.net API 요청 ├─ 파싱 (타임스탬프 추출) └─ setLyrics() 4. Audio.Sound.createAsync() - 오디오 초기화 5. sound.playAsync() - 재생 시작 6. setOnPlaybackStatusUpdate() - 진행률 추적
```

6.3 가사 동기화 흐름

```
Audio 재생 중 ↓ 250ms 마다 setCurrentMs() 호출 ↓ currentMs와 가사 라인의 startMs/endMs 비교 ↓ 현재 라인 인덱스 계산 ↓ LyricsView에서 하이라이트 및 스크롤 ↓ 가사 끝나면 자동으로 다음 곡 재생
```

6.4 단어 검색 흐름

```
사용자: 가사의 단어 클릭 ↓ handleWordPress(word) ↓ getWordDefinition(word) ├─ 정규식으로 단어 정제 └─ Free Dictionary API 호출 ↓ 응답 파싱 ↓ VocabCard로 표시 (뜻, 발음, 예문)
```

7. 화면 구조 및 메뉴

7.1 화면 네비게이션 맵

```
┌──────────────────────────────────────────┐ │ 앱 시작 │
└──────────────┬───────────────────────────┘ ↓ ┌──────────────┐ │ 로그인 확인 │
└──────┬──────┘ ↓ 토큰 있음? ├─ NO │ └→ ┌──────────────┐ │ │ LoginScreen │ │
│ [Spotify 로그인] │ │ └──────┬──────┘ │ ↓ └──────┬──────┘ ↓
┌──────────────────┐ │ HomeScreen │ │ (곡 목록) │ ┌─┤ [곡 선택] │ │
└──────────────────┘ │ ↓ ┌──────────────────┐ │ PlayerScreen │ │ (플레이어) │
│ - 가사 표시 │ │ - 플레이 컨트롤 │ │ - 단어 검색 │ ├─┤[← 뒤로] │ │ └──────────────────┘
│ └──→ HomeScreen으로 돌아가기
```

7.2 화면별 구성 요소

LoginScreen

- 앱 로고 및 설명
- 기능 소개 (3개 항목)
- Spotify 로그인 버튼
- 주의사항 (30초 미리듣기 제한)

HomeScreen

- 헤더 (제목 + 곡 개수)
- BTS 곡 목록 (FlatList)
- 각 곡 아이템:
  - 앨범 아트
  - 곡 제목
  - 앨범명
  - 재생 시간

PlayerScreen

- 배경 (앨범 아트 + 블러)

- 앨범 아트 (크게)
- 곡 정보 (제목, 아티스트)
- 가사 뷰 (스크롤 가능)
  - 현재 라인 하이라이트
  - 단어별 클릭 가능
- PlayerControls
  - 재생 바
  - 재생시간 / 전체시간
  - 이전, 재생/일시정지, 다음
- VocabCard (단어 선택 시)

---

8. 컴포넌트 명세

8.1 화면 컴포넌트

컴포넌트	파일명	Props	역할
LoginScreen	src/screens/LoginScreen.tsx	없음	Spotify OAuth 로그인
HomeScreen	src/screens/HomeScreen.tsx	tracks, currentTrack, isPlaying, onSelectTrack	곡 목록 표시
PlayerScreen	src/screens/PlayerScreen.tsx	track, lyrics, isPlaying, currentMs, callbacks	음악 재생 및 가사 표시

8.2 UI 컴포넌트

컴포넌트	파일명	기능
PlayerControls	src/components/PlayerControls.tsx	재생 컨트롤 (슬라이더, 버튼)
LyricsView	src/components/LyricsView.tsx	가사 표시 및 단어 클릭
VocabCard	src/components/VocabCard.tsx	단어 정의 카드

8.3 서비스 계층

서비스	파일명	주요 함수
Spotify Auth	src/services/spotifyAuth.ts	initiateSpotifyLogin(), handleSpotifyCallback(), getStoredToken()
Spotify API	src/services/spotify.ts	getBTSAllTracks(), getAlbumTracks(), searchBTSSongs()

Lyrics	src/services/lyrics.ts	getLyrics(), getWordDefinition(), parseSyncedLyrics()

9. 환경 설정

9.1 환경 변수 (.env)

필수 변수

```
EXPO_PUBLIC_SPOTIFY_CLIENT_ID=your_client_id_here
```

선택 변수

```
EXPO_PUBLIC_FIREBASE_API_KEY=your_firebase_api_key
EXPO_PUBLIC_FIREBASE_APP_ID=your_firebase_app_id
```

9.2 설정 파일

src/config/spotify.ts

```
export const SPOTIFY_CONFIG = { clientId:
process.env.EXPO_PUBLIC_SPOTIFY_CLIENT_ID, redirectUri:
'http://localhost:19006/', scopes: [ 'streaming', 'user-read-email', 'user-
read-private', ... ] }; export const BTS_ARTIST_ID = '3Nrfpe0tUJi4K4DXYWgMUX';
```

9.3 Spotify Developer 설정

필수 단계

1. https://developer.spotify.com/dashboard 접속
2. 앱 생성 및 Client ID 획득
3. Edit Settings에서 Redirect URIs 등록:

   ```
   http://localhost:19006/ http://localhost:3000/ http://localhost:8081/
   ```

9.4 TypeScript 설정

```
{ "extends": "expo/tsconfig.base", "compilerOptions": { "strict": true } }
```

10. 설치 및 실행 가이드

10.1 사전 요구사항

- Node.js 18+ 설치
- npm 또는 yarn 설치
- Expo CLI: `npm install -g @expo/cli`
- Spotify 계정 (무료)
- Spotify Developer 앱 생성

10.2 단계별 설치

1단계: 프로젝트 클론

```
git clone https://github.com/ymi-hub/K-pop.git cd K-pop
```

2단계: 의존성 설치

```
npm install
```

3단계: 환경 설정

```
cp .env.example .env # .env 파일에 Spotify Client ID 입력
```

4단계: Spotify 설정

1. https://developer.spotify.com/dashboard 접속
2. Your App → Edit Settings
3. Redirect URIs에 추가: http://localhost:19006/

10.3 앱 실행

웹 버전 (권장)

```
npm run web # http://localhost:19006에서 앱 열기
```

iOS

```
npm run ios # iOS 에뮬레이터에서 실행
```

Android

```
npm run android # Android 에뮬레이터에서 실행
```

11. 에러 처리 및 제한사항

11.1 주요 제한사항

항목	제한사항	이유
음악 재생	30초 미리듣기만	Spotify API 정책
로그인	웹 환경에서만	Spotify OAuth 제한
전곡 듣기	Spotify Premium 필요	Spotify 정책
가사	lrclib.net에 있는 곡만	API 의존성
네트워크	인터넷 필수	API 기반 앱

11.2 에러 처리 전략

API 호출 에러

```
try { const response = await fetch(API_URL); if (!response.ok) { throw new Error(`API Error: ${response.status}`); } return await response.json(); } catch (error) { console.error('Error:', error); // 사용자 친화적 메시지 표시 return []; }
```

토큰 만료 처리

- 만료된 토큰 자동 삭제
- 재로그인 요청
- 로그인 화면으로 리다이렉트

11.3 일반적인 문제 해결

문제	해결 방법
"Client ID Invalid"	.env 파일의 Client ID 확인 후 재입력

"redirect_uri not matching"	Spotify Dashboard의 Redirect URIs 재확인
npm install 실패	node_modules 삭제 후 npm cache clean --force 실행
가사 안 나옴	lrclib.net에 해당 곡이 없을 수 있음

12. 향후 개선 계획

12.1 단기 계획 (1-3개월)

- ✅ Spotify Web Playback SDK 통합 (전곡 재생)
- ✅ 사용자 즐겨찾기 기능
- ✅ 학습 통계 (학습한 단어 수, 재생 시간)
- ✅ 오프라인 가사 캐싱
- ✅ Shuffle/Repeat 컨트롤

12.2 중기 계획 (3-6개월)

- 🔜 다른 아티스트 추가 (BLACKPINK, Stray Kids 등)
- 🔜 사용자 커뮤니티 기능
- 🔜 AI 기반 맞춤형 학습 계획
- 🔜 플레이리스트 공유
- 🔜 Firebase 인증 통합

12.3 장기 계획 (6개월 이상)

- 🚀 다국어 지원
- 🚀 AR 기능 (가사 오버레이)
- 🚀 게임화 (학습 배지, 도전과제)
- 🚀 라이브 채팅 기능
- 🚀 안드로이드/iOS 스토어 배포

12.4 기술 개선

- 상태 관리 라이브러리 도입 (Zustand, Redux Toolkit)
- 자동화 테스트 추가

- 성능 최적화 (메모리, 렌더링)
- 접근성 개선 (스크린리더 지원)
- 다크 모드 완전 지원

부록

A. 파일 구조

```
K-pop/ ├── src/ | ├── screens/ # 화면 컴포넌트 | | ├── HomeScreen.tsx | | ├── PlayerScreen.tsx | | └── LoginScreen.tsx | ├── components/ # UI 컴포넌트 | | ├── PlayerControls.tsx | | ├── LyricsView.tsx | | └── VocabCard.tsx | ├── services/ # 비즈니스 로직 | | ├── spotify.ts | | ├── spotifyAuth.ts | | └── lyrics.ts | ├── config/ # 설정 | | ├── spotify.ts | | └── firebase.ts | ├── theme/ # 디자인 토큰 | └── types/ # TypeScript 타입 ├── .env # 환경 변수 ├── .env.example # 환경 변수 템플릿 ├── App.tsx # 메인 앱 ├── package.json # 의존성 ├── tsconfig.json # TypeScript 설정 └── SETUP.txt # 설치 가이드
```

B. 주요 TypeScript 타입

```
interface Track { id: string; name: string; artists: string[]; album: string; albumArt: string; durationMs: number; previewUrl: string | null; spotifyUri: string; } interface LyricLine { startMs: number; endMs: number; text: string; words: Word[]; } interface VocabEntry { word: string; meaning: string; pronunciation?: string; example?: string; difficulty: 'easy' | 'medium' | 'hard'; }
```

C. 참고 자료

- Spotify Web API: https://developer.spotify.com/documentation/web-api/
- lrclib.net API: https://lrclib.net/
- Free Dictionary API: https://dictionaryapi.dev/
- React Native Docs: https://reactnative.dev/
- Expo Docs: https://docs.expo.dev/
- GitHub Repository: https://github.com/ymi-hub/K-pop

K-pop English 프로젝트 정의서
작성일: 2026년 3월 25일
버전: 1.0.0

© 2026 All Rights Reserved